ORIVELLUM

ULTIMATE TRAINING MANUAL

Intelligence Platform for Research, Writing & Knowledge Management

Installation · LLM Configuration · Database Maintenance · Complete Page Guide

Version 1.0 · August 2026 · Internal Use Only

Table of Contents

Chapter 1

System Overview

Orivellum is a self-hosted, AI-powered intelligence platform built for authors, researchers, and knowledge workers who need to manage large bodies of source material, extract structured knowledge from it, and produce high-quality written work from that knowledge — all with full local-first privacy. Every AI feature runs against a local LLM server you control; no data is sent to external cloud providers.

Architecture at a Glance

Layer	Technology	Role
Backend API	Python 3.12+ · FastAPI · uvicorn	REST + streaming SSE endpoints, background pipeline, nightshift daemon
Database	SQLite (WAL mode) · FTS5 · custom migrations	All documents, knowledge, conversations, embeddings, settings
Document pipeline	pypdf / pdfplumber · Tesseract OCR · python-docx · openpyxl · markitdown	Extract, chunk, embed, harvest knowledge from every file type
AI gateway	Local OpenAI-compatible server (Lemonade or Ollama)	Chat, extraction, embeddings, TTS, ASR — all local
Web frontend	React 19 · Vite 7 · TanStack Query · Tailwind CSS 4 · Wouter	Full workspace UI — every page in this manual
Mobile app	Expo / React Native	Read-aloud, chat, library, learning — on iOS & Android
Search	SQLite FTS5 (lexical) + local embeddings (semantic) + BM25 hybrid	Retrieval-augmented generation for chat context
Web research	Tavily API · YouTube Transcript API · RRF + BM25 pipeline	Multi-mode internet research with citation assembly

Key Concepts

Work — A research or writing project. The core organizational unit. A Work links documents, knowledge items, conversations, tasks, chapters, and book pipeline state.

Document — Any uploaded file — PDF, DOCX, XLSX, CSV, Markdown, audio, image, ZIP archive. Documents are processed through the extraction pipeline to produce text, chunks, and knowledge items.

Knowledge Item — A structured fact or claim extracted from a document by rule-based and/or LLM-based harvesting. Knowledge items have confidence scores, source citations, and review status.

Conversation — An AI chat session. Can be linked to a Work so the AI draws context from that Work's knowledge when answering.

Nightshift — A background daemon that runs nightly (default 3 AM). It vacuums the database, processes stuck documents, refreshes embeddings, detects knowledge gaps, and runs 20+ maintenance passes.

MCOS — Model Calibration & Observability System. Tracks every LLM call (model, latency, tokens, success/failure), runs benchmark suites, and maintains a prompt health registry for regression detection.

Embeddings — Vector representations of text used for semantic search. Stored locally in SQLite. Requires an embedding model running on your local LLM server.

Readiness — A document's processing state: *imported* (queued), *ready* (processed), *error* (extraction failed), *no_text* (nothing readable), *draft* (manually created).

Chapter 2

Installation & Setup

Prerequisites

Orivellum has two categories of dependency: managed (installed automatically by the setup scripts) and system (must be installed by your OS package manager or the Windows setup script before running the application).

Runtime Requirements

Component	Version	Notes
Python	3.12+	3.13 works; 3.12 is the pinned version in .python-version
Node.js	20+	Required for the web frontend and mobile app
pnpm	9+	Workspace package manager — do not substitute npm or yarn
uv	0.4+	Python package manager and virtual-environment tool

System Tools (OS-level)

Tool	Purpose	Ubuntu/Debian	macOS
Tesseract OCR	OCR for scanned PDFs and images	sudo apt-get install tesseract-ocr	brew install tesseract
Poppler	PDF-to-image conversion (pdf2image)	sudo apt-get install poppler-utils	brew install poppler
FFmpeg	Audio conversion for TTS/ASR pipeline	sudo apt-get install ffmpeg	brew install ffmpeg
espeak-ng	Fallback text-to-speech (optional)	sudo apt-get install espeak-ng	brew install espeak

■■ Important

uv cannot install system tools. Tesseract and Poppler must be installed via your OS package manager (apt, brew, or the Windows setup script) BEFORE running uv sync. The diagnostics page will tell you if either is missing.

Linux / Replit — Step-by-Step Setup

Install system tools

```
sudo apt-get install -y tesseract-ocr poppler-utils ffmpeg espeak-ng
```

Clone or open the project

```
git clone <repo-url> orivellum && cd orivellum
```

Install Python packages

```
uv sync
```

Install Node packages

```
pnpm install
```

Start everything

```
./start.sh # waits for API health, then starts Vite
```

Or start manually

```
uv run python -m orivellum.api.main & PORT=5173 BASE_PATH=/
ORIVELLUM_API_URL=http://127.0.0.1:8080 pnpm --filter @workspace/orivellum-ui run dev
```

Windows — Step-by-Step Setup

Windows uses two PowerShell scripts. The setup script installs all dependencies automatically (Python 3.12, Node LTS, pnpm, uv, Tesseract 5.5.0, Poppler 24.08, FFmpeg, espeak-ng 1.51.1). The start script builds the UI and launches the API server.

Allow scripts (one-time)

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Run full setup

```
.\scripts\setup-windows.ps1
```

Start the application

```
.\scripts\start.ps1
```

Open in browser

```
http://localhost:8080/orivellum-ui/
```

Start with mobile app

```
.\scripts\start.ps1 -Mobile
```

Skip UI rebuild (faster)

```
.\scripts\start.ps1 -SkipBuild
```

Environment Variables

All variables are optional — defaults work for a local single-user installation.

Variable	Default	Description

ORIVELLUM_AI_URL	http://127.0.0.1:13305/api/v1	Base URL of your local LLM server
ORIVELLUM_DATA_DIR	./data	Where the database and library files are stored
ORIVELLUM_LOG_LEVEL	INFO	Logging verbosity: DEBUG / INFO / WARNING / ERROR
ORIVELLUM_API_KEY	(generated)	Bearer token for API authentication
ORIVELLUM_PORT	8080	Port the API server listens on
ORIVELLUM_HOST	0.0.0.0	Network interface the API binds to
ORIVELLUM_DB_PATH	(data_dir/orivellum.db)	Explicit database file path
ORIVELLUM_CONTEXT_WINDOW	32768	Token context window for chat
ORIVELLUM_EXTRACTION_TIMEOUT	30	Seconds allowed per LLM extraction call
ORIVELLUM_EMBEDDER_MODEL	(config.yaml value)	Name of the embedding model
ORIVELLUM_WORKERS	8	Thread pool size for background pipeline work
PORT	8080	Alternate port override (same as ORIVELLUM_PORT)
TAVILY_API_KEY	(none)	Required only for internet research / web search features

Chapter 3

LLM Configuration

Orivellum does not connect to any external AI API. Every AI feature — chat, document extraction, knowledge harvesting, embeddings, text-to-speech, speech-to-text, web research synthesis — uses a local OpenAI-compatible server that you run and control. Your data never leaves your machine.

Choosing a Local LLM Server

Two servers are officially tested and supported:

Server	Install	Start Command	API URL	Best For
Lemonade Server (Primary / Recommended)	pip install lemonade-server	lemonade-server --port 13305	http://127.0.0.1:13305/api/v1	AMD Ryzen AI / NPU acceleration; native Windows; recommended for the hardware this system was designed for
Ollama	ollama.com/download	ollama serve	http://127.0.0.1:11434/v1	Easy cross-platform setup; wide model library; Nvidia/CPU fallback
LM Studio	lmstudio.ai	(GUI app)	http://127.0.0.1:1234/v1	GUI-first; good for exploring models before committing
vLLM / llama.cpp server	(various)	varies	any /v1 path	Linux servers, high-throughput production use

Model Roles and Recommendations

Orivellum uses up to six named model slots. Each is configured in **config.yaml** in the project root under the serving: key. Model names must match exactly what your LLM server reports — check GET /v1/models on your server if unsure.

Slot	config.yaml key	Primary Role	Recommended Models
Workhorse	workhorse_model	Chat, summarisation, general reasoning, web research synthesis	Qwen3.6-35B-A3B-GGUF (fast MoE, vision built in) Qwen3-30B-A3B-Instruct-2507-GGUF

Reasoner	reasoner_model	Complex multi-step reasoning tasks, verification, deep analysis	gpt-oss-120b-mxfp-GGUF (best local, needs 128 GB RAM) gpt-oss-20b-mxfp4-GGUF (smaller machines)
Coder	coder_model	Code generation, document workshop, structured output	Qwen3-Coder-30B-A3B-Instruct-GGUF (256K context) Qwen3-Coder-Next-GGUF (~48 GB)
Embedder	embedder_model	Semantic search vectors for all documents and knowledge	Qwen3-Embedding-8B-GGUF (recommended) nomic-embed-text-v2-moe-GGUF (lightweight)
Vision	vision_model	OCR of scanned documents and images (fallback to Tesseract)	Qwen3.6-35B-A3B-GGUF (same as Workhorse) (leave empty to use Tesseract)
Reranker	reranker_model	Re-ranks search results for better retrieval quality (optional)	bge-reranker-v2-m3-GGUF (optional)

config.yaml — Complete Reference

Create or edit **config.yaml** in the project root. Everything is optional — comment out any line to use the default.

```
serving: base_url: "http://127.0.0.1:13305/api/v1" # Your LLM server URL workhorse_model: "Qwen3.6-35B-A3B-GGUF" # General-purpose model reasoner_model: "gpt-oss-120b-mxfp-GGUF" # Complex reasoning coder_model: "Qwen3-Coder-30B-A3B-Instruct-GGUF" # Code & structured output embedder_model: "Qwen3-Embedding-8B-GGUF" # Semantic embeddings vision_model: "Qwen3.6-35B-A3B-GGUF" # OCR (same as workhorse) reranker_model: "" # Leave empty → no reranking context_window: 131072 # Token context limit timeout_sec: 300 # LLM call timeout (seconds) extraction_timeout_sec: 120 # Per-document extraction timeout server: port: 8080 host: "0.0.0.0" api_key: "" # Leave empty → auto-generated data: dir: "./data" # Database and library storage
```

Pulling Models (Ollama Example)

Run these commands after ollama serve is running:

```
# Workhorse (MoE models are dramatically faster on unified-memory machines) ollama pull qwen3:30b-a3b # Reasoner (best local reasoning — needs 128 GB RAM) ollama pull gpt-oss:120b # Embeddings (required for semantic search) ollama pull qwen3-embedding:8b # Coder (optional — for document workshop) ollama pull qwen3-coder:30b
```

Verifying LLM Connectivity

Go to **System** → the status cards at the top of the page will show:

- **LLM Status** — green circle = connected, red = unreachable
- **Semantic Search** — shows embedding model status and circuit breaker state

- **TTS / ASR** — text-to-speech and speech-to-text availability

■■ Tip

If the System page shows the LLM as unreachable, check that your LLM server is running and that ORIVELLUM_AI_URL (or config.yaml serving.base_url) matches the URL exactly, including the /v1 path suffix.

MCOS — Model Calibration & Observability System

MCOS is Orivellum's internal telemetry and quality-assurance system for LLM calls. It runs transparently in the background — you don't need to configure it.

- **Telemetry** — every LLM call is logged: model name, purpose, latency, input/output tokens, success/failure, error
- **Prompt registry** — system prompts are versioned; regressions are detected when a prompt's quality drops
- **Benchmark suites** — nightly runs test model quality against known-correct Q&A pairs
- **MCOS page** — view call history, model health, benchmark results, and prompt health at /mcos

■ Why MCOS matters

When you switch models or update your LLM server, MCOS will immediately surface any quality regressions in extraction, chat, or search. Check the MCOS page after any major configuration change.

Chapter 4

Database Maintenance & Troubleshooting

Orivellum uses a single SQLite database file at **data/orivellum.db** (default). SQLite is reliable in WAL (Write-Ahead Logging) mode, which is how Orivellum runs it. This chapter covers routine maintenance, reprocessing stuck documents, fixing common errors, and restoring a healthy state.

Schema Versions and Migrations

The database schema is managed by an append-only migration system. Every migration is numbered, idempotent, and runs automatically when the application starts. The current schema is **v101**. You never need to run migrations manually — just start the application and they apply.

> ■■ **Tip**
>
> To check the current schema version: go to System → Diagnostics, or run: uv run python scripts/run_diagnostics.py — the report shows schema_version.

The Nightshift Daemon

Nightshift is an automatic background maintenance daemon that runs every night at 3 AM local time (configurable in Settings). It is the primary tool for keeping the database healthy — think of it as an automated DBA running nightly maintenance.

What Nightshift Does (in order)

DB Optimisation — Runs SQLite integrity_check, WAL checkpoint, ANALYZE, and conditional VACUUM (only if >30% of pages are free)

Temp cleanup — Removes zero-byte output files and orphaned .part upload temp files

Report pruning — Keeps the last 30 nightly reports in data/nightshift/

Orphan cleanup — Deletes knowledge items, chunks, and vectors whose parent documents were deleted

Stuck-document recovery — Retries up to 5 documents per run that have been stuck in 'imported' or 'error' state for more than 10 minutes

VLM re-extraction — Re-processes scanned PDFs and images that landed in no_text using the vision model (if configured)

Audio retranscription — Re-runs speech-to-text on audio files that failed or have outdated transcripts

Sparse doc harvest — Runs LLM knowledge harvest on documents that have fewer than 3 knowledge items

Gap analysis — Detects knowledge gaps in active Works and writes results to the gap cache

Evidence rescore — Re-scores knowledge item confidence and detects contradictions

Chunk-prefix backfill — Adds contextual prefixes to chunks missing them, then re-embeds

Embedding backfill — Generates embeddings for chunks and knowledge items that are missing vectors

Stats refresh — Updates work-level aggregate statistics (document count, knowledge count, etc.)

MCOS benchmarks — Runs model quality benchmarks and checks prompt health for regressions

Outbox drain — Delivers up to 500 queued background notifications

Audit chain verification — Verifies the integrity of the governance audit log

Version suggestions — Suggests version relationships between similar documents in the same Work

Topic clustering — Groups knowledge items into topic clusters for the Topics page

Memory management — Expires old working memory, deduplicates, promotes episodic→semantic memory

Triggering Nightshift Manually

You can trigger a full nightshift run immediately without waiting for 3 AM:

- Go to **System** → Nightshift card → click **Run Now**
- Or via API: POST /api/system/nightshift/run-now
- Check the result in System → Nightshift → last report

Document Readiness States

Every document has a **readiness** field that shows where it is in the pipeline:

State	Meaning	What to Do
imported	Queued for extraction — waiting for the pipeline to pick it up	Wait up to 2 minutes. If it stays 'imported' for more than 10 minutes, the pipeline may be stuck — use Reprocess.
ready	Fully extracted, chunked, embedded, and knowledge-harvested	Nothing — this is the healthy terminal state.
error	Extraction failed with an error message	Click the document → see the error detail → fix the root cause (missing tool, corrupt file) → click Reprocess.
no_text	File processed but no readable text found	For scanned PDFs: configure a vision model. For images: check Tesseract is installed. Then Reprocess.
draft	Manually created document placeholder	Add content or link a real file.

Reprocessing Documents

Single Document

- Go to **Library** → click the document → click **Reprocess** in the document detail header
- Or via API: POST /api/library/{doc_id}/reprocess
- Use ?force=true to reprocess even if the document is already ready

Bulk Reprocess (All Stuck Documents)

The reprocess-all endpoint fixes every document that is stuck, in error, or has no text in one call:

```
POST /api/library/reprocess-all
```

This endpoint:

- Resets all 'imported', 'error', and 'no_text' documents back to the 'imported' state
- Handles ZIP archives that haven't been exploded yet
- Re-queues everything for the extraction pipeline
- Returns a count of how many documents were re-queued

From the Command Line

```
# Run the full diagnostics report (identifies every problem) uv run python
scripts/run_diagnostics.py # Run diagnostics + SQLite VACUUM (compacts the database) uv run
python scripts/run_diagnostics.py --vacuum
```

Repairing Embeddings

Embeddings must be rebuilt any time you change the embedding model or its output dimension. Running a mismatched model produces wrong semantic search results without any obvious error.

- Go to **System** → Semantic Search card → click **Probe Embeddings**
- If the probe succeeds but dimensions don't match: click **Reindex Embeddings**
- Reindex deletes all stored vectors, verifies the embedder is live, then rebuilds from scratch
- Large libraries may take several hours — search falls back to BM25 lexical matching during this time

■ ■ Important

Do not run a VACUUM on a database that has failed an integrity_check. A corrupted database may appear to compact but the corruption is preserved. Restore from a backup first, then vacuum the restored copy.

Common Problems and Fixes

Problem: Document stuck in 'imported' forever

Fix: The pipeline thread may have crashed. Trigger Reprocess on the document, or use reprocess-all. Check the API server logs for exceptions. Nightshift will automatically retry stuck documents (>10 min) each night.

Problem: All AI features fail (chat won't respond, extraction fails)

Fix: Your LLM server is not running or the URL is wrong. Check ORIVELLUM_AI_URL / config.yaml serving.base_url. Verify the server is running: curl http://127.0.0.1:13305/v1/models (Lemonade) or curl http://127.0.0.1:11434/v1/models (Ollama).

Problem: Semantic search returns poor results or nothing

Fix: Embeddings may be missing or dimension-mismatched. Go to System → Semantic Search → Probe → Reindex. Check that your embedder model name in config.yaml exactly matches what the server reports.

Problem: PDFs come back as no_text (scanned documents)

Fix: Tesseract is either not installed or the PDF is image-only. Install tesseract-ocr and poppler-utils, then Reprocess. For better quality, configure a vision_model in config.yaml.

Problem: Database grows very large (>1 GB)

Fix: Run nightshift (which does a conditional VACUUM) or: uv run python scripts/run_diagnostics.py --vacuum

Problem: Schema migration fails on startup

Fix: The database file may be from a newer version or be corrupt. Check the error message in the API logs. If corrupt, restore from the most recent backup in data/backups/.

Problem: Knowledge harvest produces no items

Fix: AI extraction may be disabled. Go to Settings → check 'AI Extraction Enabled'. Also verify the workhorse_model is set and reachable.

Problem: Nightshift hasn't run in more than 2 days

Fix: Go to System → Nightshift card → Run Now. Check if the last run had an error. API server must be running at the nightshift hour (default 3 AM).

Chapter 5

Page-by-Page User Guide

This chapter covers every page in the Orivellum web application. The navigation sidebar is always visible on the left. Pages are grouped by workflow: Knowledge → Creation → Learning → Administration.

Dashboard (/)

The Dashboard is the command centre of your Orivellum workspace. It gives you a complete picture of your active projects, recent activity, and what needs attention — all on one screen.

Sections

- **Custodian Header** — time-of-day greeting, pending task count, and quick-action buttons
- **Suggestions Panel** — AI-generated suggestions for your active Work (tasks, research questions, gaps to fill)
- **Gap Analysis Panel** — knowledge gaps detected across your Works, with suggested actions
- **Scorecard** — aggregate health metrics: total documents, knowledge items, pending reviews, active conversations
- **Active Works Cards** — your in-progress Works with progress bars and quick navigation
- **Recent Documents** — last-uploaded or last-modified documents with readiness indicators
- **Recent Conversations** — your most recent AI chat sessions with model and timestamp
- **Activity Feed** — timestamped log of everything that has happened across the system

How to Use

- Start here every session — the gap panel and suggestions tell you what to work on next
- Click any Work card to jump directly to that work's detail page
- The scorecard turning yellow or red on any metric means something needs attention

Works (/works)

Works are your top-level research or writing projects. Every document, knowledge item, conversation, and task belongs to a Work. Think of a Work as a book project or major research topic.

Sections

- **Page Header** — Create Work button, search bar, type/status filters
- **Works Grid** — cards showing title, type, status, document count, knowledge count, and a completion progress bar

Work Types

- **Research** — for academic or investigative projects
- **Fiction / Narrative** — for novels and long-form creative writing
- **Non-fiction** — for books, essays, journalism
- **Reference** — for knowledge bases, wikis, training material

How to Use

- Click **Create Work** → fill in title, type, and optional description → Save
- Click any Work card to open the full Work workspace
- Use the filter chips to show only Fiction, Research, or a specific status

Work Detail (/works/:workId)

The Work Detail page is the main workspace for a single project. It combines every aspect of managing a Work — documents, knowledge, chapters, tasks, conversations, health monitoring, and AI tools — all in a tabbed interface.

Tabs

- **Overview** — Work metadata, description, statistics bar (document count, knowledge count, conversations), book health card, and quick-chat button
- **Documents** — all documents linked to this Work with readiness states, upload button, and unlink option (hover a doc to reveal the X button)
- **Knowledge** — all extracted knowledge items with confidence scores, review status (AI-auto vs rule-based), approve/reject thumbs, and search
- **Book** — the book production pipeline: chapter ordering, pipeline state machine (Draft → Research → Writing → Editing → Final), and the book intelligence view
- **Brainstorm** — AI brainstorming canvas for generating ideas, outlines, and research questions linked to this Work
- **Tasks** — task list linked to this Work; create, complete, and prioritise tasks
- **Intelligence** — knowledge quality dashboard: gaps, contradictions, confidence distribution, chapter coverage

Book Health Card

The Overview tab shows a 'Book Health' card that polls automatically as documents are processed. It shows overall health score, gaps, confidence, and alerts for contradictions or missing evidence. Green = healthy, amber = needs attention, red = significant issues.

How to Use

- Upload documents via the Documents tab → they are automatically linked to this Work
- Review AI-extracted knowledge in the Knowledge tab — approve good items, reject bad ones
- Use Quick Chat (Overview tab) to open a conversation pre-linked to this Work
- Advance the book pipeline (Book tab) as you complete each production stage

Work Intelligence (/works/:workId/intelligence)

A dedicated analytics view for one Work, showing the quality and completeness of its knowledge base in depth.

Sections

- **Confidence Summary** — distribution of knowledge item confidence scores (high / medium / low)
- **Health Score** — aggregate health: completeness, evidence quality, contradiction rate
- **Knowledge Gaps** — topics that are mentioned but not well-evidenced, with suggested research questions
- **Contradictions** — pairs of knowledge items that say conflicting things
- **Chapter Coverage** — for books: which chapters have strong knowledge backing vs thin coverage
- **Knowledge Item List** — all items filterable by confidence band, source, and review status

How to Use

- Run 'Rescore Knowledge' when you've added new documents — updates all confidence scores
- Use the Gaps list to decide what to research next
- Resolve contradictions by reviewing source documents and approving/rejecting competing claims

Chat (/chat)

The Chat page is a full AI conversation workspace. Conversations can be standalone or linked to a specific Work, in which case the AI automatically draws context from that Work's knowledge base when answering.

Sections

- **Conversations Sidebar** — list of all conversations with timestamps, archived state, and rename/archive actions
- **Model & Mode Controls** — choose the active model, toggle Deep Mode (uses the Reasoner model), connectivity indicator
- **Message Thread** — streaming AI responses with markdown rendering, code syntax highlighting, and copy-to-clipboard
- **Composer** — message input with file attachment, internet search toggle, and send button
- **Sources Panel** — knowledge items and web sources that were injected as context for the current response
- **Work Badge** — when the conversation is linked to a Work, a badge links back to that Work's detail page
- **Files Panel** — documents attached to or referenced in the conversation
- **Project Compass** — shows active Work metadata and a summary of the most relevant knowledge items

Linking a Conversation to a Work

- Open a conversation → click the Work Badge selector at the top → choose a Work
- The AI will automatically inject the top 8 most relevant knowledge items from that Work with every response
- The fastest way: in the Work Detail page, click 'Quick Chat' — creates a conversation already linked

Internet Search

- Toggle the globe icon in the composer to enable web search mode
- The system automatically detects biblical/theological queries and routes them to curated sources
- YouTube video transcripts, academic papers, and Facebook public pages are all searchable
- Requires TAVILY_API_KEY to be set in environment

Deep Mode

- Toggle the brain icon to switch from the Workhorse model to the Reasoner model
- Use for complex analysis, multi-step reasoning, and evaluation tasks
- Slower but significantly more capable for hard questions

Library (/library)

The Library is the central document store. Every file you upload lives here, regardless of which Work it belongs to. The Library gives you full visibility into extraction status, knowledge, and document relationships.

Sections

- **Header** — Upload button, search bar, filter chips (by readiness, Work, file type)
- **Document Cards / Table** — shows filename, type icon, readiness badge, knowledge count, linked Work, and upload date
- **Work Filter Chip** — click to filter to only documents in a specific Work

Supported File Types

- **PDF** — 3-tier extraction: pdfplumber (text) → pdf2image+Tesseract (OCR) → vision model
- **DOCX** — full text + tables extraction via python-docx
- **XLSX / CSV** — up to 5,000 rows per sheet via openpyxl
- **PPTX** — slide text and speaker notes
- **Markdown / Text / HTML / JSON / Code** — direct text extraction
- **MP3 / WAV / M4A / FLAC** — speech-to-text via Whisper / faster-whisper
- **PNG / JPG / WEBP** — Tesseract OCR or vision model
- **ZIP** — exploded into individual child documents automatically

How to Upload

- Click **Upload** → drag files or click to browse → optionally assign to a Work before uploading
- ZIP files are automatically exploded — all contained files become individual library documents
- Duplicate detection: re-uploading the same file navigates to the existing document (SHA-256 dedup)

After Uploading

- Readiness starts at 'imported' → moves to 'ready' in 30–120 seconds depending on file type and size

- Poll for status: Library cards auto-refresh every 4 seconds while any document is in 'imported' state

Document Detail (/library/:docId)

The full inspector for a single document. Everything extracted from a document is visible and actionable here.

Sections

- **Document Header** — title, readiness badge, Reprocess button, Download button, Read Aloud button
- **Extraction Status** — warning messages from the pipeline if extraction had any issues
- **Overview Tab** — metadata (file type, size, sha256, upload date, linked Work selector, lifecycle state)
- **Knowledge Tab** — all AI-extracted and rule-based knowledge items with approve/reject review controls
- **Chunks Tab** — raw text chunks used for search and retrieval (with their embeddings status)
- **Chapters Tab** — for books processed with chapter detection: individual chapter records
- **Read Aloud Panel** — text-to-speech playback of the document's extracted text
- **Version Snapshots** — if multiple versions of this document exist, navigate between them
- **Related Documents** — near-duplicate detection results and version suggestions

Reviewing Knowledge Items

- AI-extracted items (from the LLM) show a purple Sparkles badge — review carefully
- Rule-based items (from regex/pattern matching) are reliable and rarely need review
- Click ■ to approve an item — it is then included in chat context
- Click ■ to reject — the item fades to 50% opacity and is excluded from chat context
- Rejected items can be un-rejected by clicking ■ again

Projects (/projects)

Projects are higher-level containers that group related Works together. A Project might be a book series, a research programme, or a client engagement.

Sections

- **Header** — Create Project button
- **Project Cards** — title, description, Work count, document count, concept count

Project Detail (/projects/:projectId)

- **Header** — project title, description, aggregate metrics
- **Summary Cards** — Works in project, total documents, concepts, mastery progress
- **Concepts List** — learning concepts associated with this project with mastery labels

Books (/books)

The Books page groups your Works by their book-production status. It is the high-level production overview when you're running multiple book projects simultaneously.

Sections

- **Active Books** — Works with an active book pipeline in progress, showing stage, chapter count, and progress
- **In Progress** — Works that have started the book pipeline but are at an early stage
- **Go to Works CTA** — when no books exist yet, prompts you to create a Work

Finishing (/finishing)

The Finishing page is the final production stage — where a completed manuscript gets prepared for distribution. It handles pre-flight checks, chapter ordering, export packages, cover versioning, and brand tokens.

Sections

- **Book Selector** — choose which Work/book you're finishing
- **Pre-flight Card** — automated checklist that must pass before export (all chapters present, no missing content, health score above threshold)
- **Book Metadata Card** — title, subtitle, author, ISBN, publication date
- **Chapters Card** — final chapter ordering table with drag-to-reorder and word count per chapter
- **Export Card** — generate DOCX or PDF export packages for test readers or publishers; recipient management and sign-off tracking
- **Product Spec** — physical or digital product specifications (trim size, paper type, format)
- **Cover Versions** — manage cover artwork versions
- **Seal Cover** — lock a cover version as the final canonical cover
- **Brand Tokens** — typography, colour palette, and design tokens for the book's visual identity

Learn (/learn)

The Learn page is a guided study environment. It uses Socratic questioning and spaced repetition to help you deeply understand the content in your library — not just store it.

Sections

- **Learning Header** — current streak, session progress, daily goal
- **Topic / Course Controls** — filter by Work, concept, or search for a specific topic
- **Prerequisite Cards** — learning concepts you should understand before tackling others, shown with mastery level
- **Study Panel** — current question or concept with Socratic follow-ups; session limited to 5 questions by default
- **Progress Cards** — mastery bars per concept (novice → developing → proficient → mastery)

How Socratic Learning Works

- The system asks you an open-ended question about a concept in your library
- Your answer is evaluated by the AI against known evidence from your knowledge base
- A follow-up question probes deeper or corrects a misunderstanding
- After 5 questions, the session ends and mastery is updated
- Concepts with low mastery appear at the top of your queue next session

Intake (/intake)

The Intake page is the structured starting point for a new Work. Instead of uploading raw files directly, Intake lets you define the Work's profile, source material, and extraction options before processing begins.

Sections

- **Profile Card** — Work title, type, description, and writing goals
- **Source Selection** — file upload area specifically for the intake batch
- **Metadata Fields** — tags, author, publication date, subject area
- **Extraction Options** — choose extraction depth, enable/disable AI harvest, select relevant knowledge domains
- **Submit Area** — submit the intake for processing; status shows extraction progress

Topics / The Web (/topics)

The Topics page visualises the connected web of topics that have been extracted across all your knowledge items. It reveals thematic relationships between Works and documents you might not have noticed.

Sections

- **Search / Filter** — find a specific topic or filter by Work
- **Topic Cards** — each cluster of related knowledge items is grouped into a topic card with a name and summary
- **Related Connections** — clicking a topic shows which Works, documents, and other topics it connects to

Knowledge Graph (/graph)

An interactive visual graph showing entities (people, places, organisations, concepts, events) extracted from your documents and the relationships between them.

Sections

- **Graph Canvas** — force-directed node-link diagram; drag nodes to rearrange, scroll to zoom
- **Entity Kind Filter Chips** — show/hide People, Places, Organisations, Events, Concepts by type

- **Graph Legend** — colour key for entity kinds and relationship types
- **Search** — find and highlight a specific entity in the graph
- **Selection Panel** — click a node to see all knowledge items connected to that entity

How to Use

- Use filter chips to reduce visual noise when the graph is dense
- Click a node → the selection panel shows all facts linked to that entity
- Filters don't move nodes — layout stays stable between filter changes

Actions (/actions)

The Actions page is a general-purpose AI task runner. Actions are parameterised AI operations — you select a Work, fill in the action's form fields, and execute it.

Sections

- **Action Selector** — dropdown of available actions (summarise, gap analysis, generate outline, etc.)
- **Work Selector** — the Work the action should run against
- **Dynamic Input Form** — each action has its own schema of required inputs
- **Execution Controls** — Run button; status indicator while running
- **Output Panel** — result of the action, rendered as markdown
- **Action History** — previous runs with their outputs

MCOS (/mcos)

The MCOS (Model Calibration & Observability System) page is the administration panel for monitoring AI quality. Use it after changing models or to investigate unexpected AI behaviour.

Sections

- **Overview / Status Header** — total LLM calls, average latency, error rate, most recent benchmark result
- **LLM Call Log** — searchable table of every AI call: timestamp, purpose, model, tokens in/out, latency, success/failure
- **Model Health Cards** — per-model health summary: uptime, p50/p95 latency, token throughput
- **Benchmark Results** — nightly quality tests with pass/fail and regression indicators
- **Prompt Registry** — view and health-check system prompts; prompts that regressed are flagged
- **Policy / Settings** — enable/disable MCOS features, set benchmark schedule

Governance (/governance)

The Governance page is the quality-control centre for knowledge extracted by the AI. Everything the LLM extracts goes into a review queue here before it influences your research or chat context — unless you've approved automatic acceptance in Settings.

Sections

- **Review Queue Tabs** — Knowledge Items, Document Findings, Benchmark Regressions — each with a count badge
- **Unverified Items List** — AI-extracted claims waiting for review, sorted by confidence (lowest first by default)
- **Item Detail** — full text, source document, extraction method, confidence score, and provenance chain
- **Confidence Explanation** — hover any confidence score to see what drove it up or down
- **Approve / Reject Controls** — accept a claim into the knowledge base, or reject it with an optional note
- **Edit** — correct the claim text before approving

Workflow

- Items appear here automatically after LLM extraction — you don't need to trigger this
- Approve high-confidence items in batch; review low-confidence items carefully
- Rejected items are soft-deleted — they remain in the database for audit purposes

Review (/review)

The Review page is a simplified review queue focused on document findings and knowledge items that need human verification. It is similar to Governance but presents items in a card-based workflow optimised for fast review.

Sections

- **Queue Header** — total pending count, filter by Work or document
- **Filter Controls** — filter by knowledge type, confidence band, or source document
- **Pending Knowledge Cards** — one card per item showing claim, source, confidence, and Accept/Reject buttons
- **Item Detail** — expand a card to see full provenance including which chunk of which document generated the claim

System (/system)

The System page is the administration console for the Orivellum installation. This is where you check connectivity, run diagnostics, manage embeddings, configure nightshift, and review system health.

Key Cards

- **LLM Status** — live connectivity check against your LLM server; shows model name and latency
- **Semantic Search / Embeddings** — embedding model health, circuit breaker state, Probe and Reindex buttons

- **Diagnostics** — run the full diagnostic suite: schema, integrity, orphans, connectivity, OCR/FFmpeg, data quality — results in categorised OK/WARN/ERROR format
- **Nightshift** — last run time and result, Run Now button, next scheduled time
- **Audio Enhancement** — enable/disable DeepFilterNet3 noise reduction for audio files
- **Settings** — enable/disable AI extraction, nightshift, deep-mode defaults, and other system-wide toggles
- **API Key** — view or regenerate the bearer token used by external API clients and the mobile app

When to Visit

- After installing: verify LLM connectivity and embeddings are green
- After changing models: run Diagnostics and Probe Embeddings
- When documents stop processing: check Diagnostics for 'stuck documents' warning
- Monthly: trigger Nightshift Run Now and check the report

Studio (/studio — in Works Detail)

The Studio is an AI document generation workspace embedded inside the Work Detail page. It lets you instruct the AI to generate new documents — chapters, outlines, summaries, reports — using your knowledge base as the source of truth.

Sections

- **Plan Panel** — describe what you want to generate; the AI creates a structured writing plan
- **Execute Panel** — approve the plan and run generation; streams the document in real time
- **Critique Loop** — the AI critiques its own output against your knowledge base, then revises
- **Output Panel** — final document with export to DOCX, Markdown, or direct library import
- **Read Aloud** — listen to the generated document via TTS

Chapter 6

Mobile App

The Orivellum mobile app (iOS and Android) is an Expo/React Native companion to the web platform. It focuses on the most portable workflows: reading documents aloud, chatting with your AI, reviewing knowledge, and learning on the go. It connects to the same local API server as the web app.

Connecting the Mobile App

- Open the mobile app → Settings → API URL → enter your server's URL
- On the same Wi-Fi: use your machine's local IP address (e.g. http://192.168.1.100:8080)
- Remotely: use Tailscale or a similar VPN — the API supports CORS for the 100.64–127.x.x Tailscale range
- The API key is shown on the System page — enter it in the mobile app's Settings → API Key field

Mobile Pages

Library — Browse, search, and upload documents. Tap a document to open the detail view with knowledge review and Work linking.

Read Aloud — Listen to any document in chunked TTS. Full playback controls, silence mode support. Downloads the audio file to device storage.

Chat — Full AI conversation with streaming responses. Linked to any Work — same context injection as the web app.

Works Detail — Overview, Documents, Knowledge, and Brainstorm tabs for a Work — same content as the web but optimised for touch.

Learn — Socratic learning sessions on the go. Same question-answer-feedback loop as the web.

Knowledge Graph — Touch-interactive entity graph with the same filter chips and entity colours as the web.

■■ Important

The 'Save to Files' button for downloaded audiobooks is only shown on iOS, not on Android — Android saves to a system-accessible location automatically.

Chapter 7

Workflows & Best Practices

Starting a New Book Project — Full Workflow

Step 1: Create a Work

Works → Create Work → set title, type (Fiction/Non-fiction), description → Save

Step 2: Upload source material

In the Work Detail → Documents tab → Upload. Add all research PDFs, DOCX, notes, and reference files. ZIPs are automatically exploded.

Step 3: Wait for extraction

Documents tab shows readiness progress. All documents should reach 'ready' within a few minutes. Reprocess any that error.

Step 4: Review AI knowledge

Work Detail → Knowledge tab → review AI-extracted items (purple Sparkles badge). Approve good ones, reject bad ones. Also check Governance for the full queue.

Step 5: Check the intelligence report

Work Detail → Intelligence tab. Note any gaps or contradictions. Add missing research documents.

Step 6: Start writing

Use the Brainstorm tab to generate outlines and ideas. Use Studio (inside Book tab) to generate chapter drafts.

Step 7: Use Chat for research questions

Quick Chat → ask questions about your topic. The AI draws from your approved knowledge items.

Step 8: Advance the book pipeline

Work Detail → Book tab → advance the state machine (Draft → Research → Writing → Editing → Final) as each stage completes.

Step 9: Finishing

When writing is complete: Finishing page → run pre-flight → order chapters → export DOCX for your editor.

Research Workflow (Non-book)

- Create a Work with type 'Research'
- Upload all source PDFs and documents
- Enable AI extraction in Settings (if not already on)
- Use the Knowledge Graph to spot entity relationships you hadn't noticed
- Use the Topics page to see which themes cluster across your sources

- Chat with your sources: 'What does the literature say about X?' draws from your approved knowledge
- Use internet search in Chat for gaps not covered by your documents

Daily Maintenance Checklist

- Dashboard → check gap analysis for new gaps detected overnight
- Governance → review any new AI-extracted items in the queue
- Library → check for any documents stuck in 'error' or 'no_text' state
- System → confirm LLM status is green

Monthly Maintenance Checklist

- System → Nightshift → Run Now — verify the report shows no critical errors
- System → Diagnostics → Run — review any WARN or ERROR items
- System → Embeddings → Probe — confirm embeddings are healthy
- If the database is over 500 MB: System → Diagnostics → Run with --vacuum flag

Search Tips

- Library search uses hybrid FTS5 + semantic search — short queries (<3 words) use lexical only
- For best semantic results: phrase your search as a complete question, not just keywords
- In Chat: link the conversation to a Work before asking — the AI searches that Work's knowledge base
- Knowledge items tagged as 'rejected' are excluded from search and chat context

Internet Research — Search Modes

When internet search is enabled in Chat, the system automatically selects the best search mode:

- **WEB** — always included; general web results
- **BIBLICAL** — auto-detected from scripture/theology keywords; searches biblegateway, biblehub, blueletterbible, ccel, thegospelcoalition, and 17 other curated sources
- **NEWS** — auto-detected from words like 'today', 'latest', '2026'; uses Tavily news lane
- **YOUTUBE** — searches YouTube and pulls full video transcripts (not just titles)
- **FACEBOOK** — searches publicly-indexed Facebook pages only
- **ACADEMIC** — restricts to arxiv, PubMed, JSTOR, Semantic Scholar, and similar scholarly sources

Chapter 8

Quick Reference

API Endpoints — Most Useful

Endpoint	Method	Description
`/api/library/upload`	POST	Upload a file (multipart/form-data). Fields: file, work_id (optional)
`/api/library/reprocess-all`	POST	Bulk reprocess all stuck/error/no_text documents
`/api/library/{doc_id}/reprocess`	POST	Reprocess a single document. Add ?force=true to force even if ready
`/api/system/nightshift/run-now`	POST	Trigger a full nightshift maintenance run immediately
`/api/system/nightshift/status`	GET	Last run time, result, and next scheduled time
`/api/system/embeddings/status`	GET	Embedding model health, dimension, vector counts
`/api/system/embeddings/probe`	POST	Live-test the embedder and reset circuit breaker on success
`/api/system/diagnostics`	GET	Full system health report (schema, integrity, tools, LLM)
`/api/works`	GET	List all Works. Query: ?work_type=&status;=
`/api/works/{id}/stats`	GET	Document count, knowledge count, conversation count for a Work
`/api/library`	GET	List documents. Query: ?work_id=&readiness;=&kind;=
`/api/conversations`	GET	List conversations. Query: ?work_id=&archived;=false

Keyboard Shortcuts (Web App)

Shortcut	Action
Ctrl + Enter	Send chat message
Ctrl + K	Open global search
Ctrl + Shift + A	Toggle archived conversations
Escape	Close any open dialog or panel
Ctrl + C (on AI message)	Copy message to clipboard (click copy button on the message)

Useful Scripts

Full system health report

```
uv run python scripts/run_diagnostics.py
```

Health report + SQLite VACUUM

```
uv run python scripts/run_diagnostics.py --vacuum
```

Re-import the Excel Training Vault

```
uv run python scripts/import_excel_vault.py
```

Start the API server manually

```
uv run python -m orivellum.api.main
```

Start the web frontend (dev mode)

```
pnpm --filter @workspace/orivellum-ui run dev
```

Start the mobile app (Expo)

```
pnpm --filter @workspace/mobile run dev
```

Run all Python tests

```
uv run pytest tests/
```

Glossary

ASR — Automatic Speech Recognition — speech-to-text using Whisper or faster-whisper

BM25 — Best Match 25 — a lexical relevance scoring algorithm used for passage ranking

Circuit Breaker — A fault-tolerance pattern that stops retrying a failing service after N failures and retries automatically after a cooldown

Embeddings — Dense numerical vectors representing text meaning, used for semantic similarity search

FTS5 — SQLite's built-in full-text search engine used for fast keyword search

LLM — Large Language Model — the AI model that powers chat, extraction, and synthesis

MCOS — Model Calibration & Observability System — Orivellum's LLM telemetry and quality framework

Nightshift — The nightly maintenance daemon that keeps the database healthy

PKLOS — Policy Knowledge and Lifecycle Operating System — Orivellum's governance and audit framework

RAG — Retrieval-Augmented Generation — injecting retrieved knowledge items as context before asking the LLM to answer

RRF — Reciprocal Rank Fusion — combines ranked lists from multiple searches into one fused ranking

SSE — Server-Sent Events — how the streaming chat responses are delivered to the browser

TTS — Text-to-Speech — converts document text to audio using the configured TTS model

WAL — Write-Ahead Logging — SQLite mode that allows concurrent reads while a write is in progress

Work — The top-level project container that groups documents, knowledge, conversations, and tasks

Orivellum Intelligence Platform

Ultimate Training Manual — Version 1.0 — August 2026
Confidential — Internal Use Only

This document was generated automatically from the live Orivellum codebase.
Re-run scripts/generate_training_manual.py to get a current version.